

BizFormPlatform

B端企业入驻订单聚合系统 — 技术实现报告

FastAPI

Next.js 16

PostgreSQL/PostGIS

Redis

Geohash

WebSocket

Docker

2026年6月 · 14万+ 测试数据 · 41 自动化测试 · github.com/freezetheflame/B-demo

1. 项目概述与架构

定位：B端企业入驻场景下的表单提交流程管理与地理批量聚合系统。核心理念：B端系统优先保障数据准确性与处理效率。

技术栈

层	技术
Frontend	Next.js 16 (React 19) + TypeScript + TailwindCSS + Zustand (MVVM) + react-window v2
Backend	FastAPI + SQLAlchemy 2.0 Async + Pydantic v2
Database	PostgreSQL 16 + PostGIS 3.4 (GIST 空间索引)
Cache	Redis 7
GIS	Geohash 编码 + Haversine 距离 + 连通分量聚类
Infra	Docker Compose

数据规模

表	记录数
merchant_forms	140,000
listing_forms	30,000
product_forms	30,000
submission_logs	200,000+

2. 前端：MVVM 模式 + 自动地理聚合

需求：“使用 react/nodejs 技术栈，结合 MVVM 模式对企业方提交的申请入驻订单根据地理坐标，自动完成批量聚合”

MVVM 架构 (Zustand)

Model 层：所有状态 (formType, page, rows, total, cache, loading) 集中在 Zustand store。ViewModel：computed 属性 (totalPages, cacheSize)。View：FormsPage 仅做渲染 + 事件绑定。

```
// store/index.ts - MVVM Model + ViewModel
export const useFormStore = create((set, get) => ({
  // — Model —
  formType: "merchant", page: 1, pageSize: 100,
  keyword: "", filterStatus: "",
  rows: [], total: 0, loading: false, error: null,
  cache: new Map(),

  // — ViewModel (computed) —
  totalPages: () => Math.ceil(get().total / get().pageSize),
  cacheSize: () => get().cache.size,

  // — Actions —
  setType: (t) => set({ formType: t, page: 1 }),
  fetchForms: async () => {
    const { formType, page, keyword, filterStatus, cache } = get();
    const cacheKey = `${formType}:${page}:${keyword}:${filterStatus}`;
    const cached = cache.get(cacheKey);
    if (cached) { set({ rows: cached.data, total: cached.total }); return; }
    // ... fetch from API, store in cache with LRU eviction
  },
}));

// View 层 - 只做数据绑定, 不包含业务逻辑
function FormsPageInner() {
  const store = useFormStore();
  useEffect(() => { store.fetchForms(); }, [store.formType, store.page]);
  return <List rowCount={store.rows.length} rowComponent={Row} />;
}
```

自动聚合（后端 —— 商户提交即触发）

```
# forms.py - 商户提交自动聚类
async def _auto_cluster_new_merchant(db, form, lat, lng):
    # ① PostGIS ST_DWithin 空间索引快速查找 500m 内候选
    result = await db.execute(sa_text("""
        SELECT id, ST_Y(geo) AS lat, ST_X(geo) AS lng
        FROM merchant_forms
        WHERE status = 'SUBMITTED' AND geo IS NOT NULL
    ""
```

```

        AND id != :my_id
        AND ST_DWithin(geo,
            ST_SetSRID(ST_MakePoint(:lng, :lat), 4326), 0.005)
    LIMIT 100
    """), {"my_id": form.id, "lat": lat, "lng": lng})

# ② Haversine 精确过滤 (消除投影误差)
for row in result.fetchall():
    if haversine_m(lat, lng, row.lat, row.lng) <= 500:
        nearby_ids.append(row.id)

# ③ 分配同一 batch_id 形成簇
batch_id = f"auto_{int(time.time())}_{form.id}"
await db.execute(sa_text(
    "UPDATE merchant_forms SET batch_id = :bid WHERE id = ANY(:ids)"
), {"bid": batch_id, "ids": nearby_ids})

return {"auto_clustered": True, "cluster_size": len(nearby_ids)+1}

```

提交 → **PostGIS** 空间索引过滤 → **Haversine** 精确匹配 → 自动归入邻近簇

3. 前端：14w+ 数据性能优化

需求："模拟并优化 10w+ 表单的聚合效率（查询和渲染性能），注意前端的缓存与网络请求"

虚拟滚动 (react-window v2)

14万条仅渲染 ~18 个 DOM 节点（14 可见 + 4 overscan），60fps。

```
// react-window v2 API - 适配自 v1 的 Breaking Changes
import { List } from "react-window";
const RowRenderer = ({ index, style }) => {
  const row = store.rows[index];
  return (<div style={style} className="flex items-center">...</div>);
};
<List style={{ height: 500 }} rowCount={rows.length}
  rowHeight={36} rowComponent={RowRenderer} rowProps={{}} />

// v1→v2 迁移: FixedSizeList→List, itemCount→rowCount,
// itemSize→rowHeight, children→rowComponent, 新增必填 rowProps
```

LRU 分页缓存

```
// 四维 cache key: {form_type}:{page}:{keyword}:{status}
cache.set(cacheKey, { data, total });
if (cache.size > 50) {
  cache.delete(cache.keys().next().value); // 驱逐最早条目
}
headers: { "Cache-Control": "max-age=30" } // HTTP 缓存兜底
```

实测性能

场景	数据量	耗时 / 帧率
表单列表查询	140,000 条	< 50ms (LRU hit)
虚拟滚动渲染	100 行/页	60fps (18 DOM nodes)
地理聚合	9,087 条	3.69s (Geohash 5 + Haversine)

4. 表单提交成功率统计

需求：“统计企业方表单提交的成功率”

核心设计：submission_logs 记录提交事件（start/success/fail/timeout），区分「提交成功率」与「审核通过率」。

```
# 后端：每次提交自动写埋点
def _log_submission(db, form_type, merchant_id, status, duration_ms, error_code):
    db.add(SubmissionLog(
        form_type=form_type, status=status,
        duration_ms=duration_ms, error_code=error_code,
        created_at=datetime.utcnow()))
```

API: GET /api/v1/analytics/submit-metrics

指标	说明
total_submits / success_count / fail_count	总量 + 成功/失败
success_rate	成功率 (%)
avg/min/max_duration_ms	耗时分布
fail_reasons	FORM_001(字段校验失败) DB_001(数据库超时) NET_001(网络超时) AUTH_001(权限)
hourly_trend	168 小时逐小时趋势

5. 知识文档快存储系统

需求："将表单改为快存储，请给出程序设计（含增删改查等逻辑）"

双表模型

KnowledgeDoc	KnowledgeDocVersion (快照表)
├ id (PK)	├ doc_id (FK)
├ title	├ version
├ content	├ content (创建/更新时的完整快照)
├ version (自增)	├ operated_at
├ status (draft published archived)	
└ deleted_at (软删除)	

完整 **CRUD** + 版本回退

操作	端点	行为
创建	POST /docs	自动 v1 + 快照
列表	GET /docs	keyword/category + 分页 + 软删除过滤
详情	GET /docs/{id}	完整 content + 版本历史
更新	PUT /docs/{id}	version+1 + 新快照
软删除	DELETE /docs/{id}	deleted_at + archived
回退	POST /docs/{id}/rollback/{v}	从快照恢复，回退本身创建新版本

```
# knowledge.py - 版本回退
@router.post("/docs/{doc_id}/rollback/{target_version}")
async def rollback_doc(doc_id, target_version, db):
    doc = await db.get(KnowledgeDoc, doc_id)
    snapshot = (await db.execute(select(KnowledgeDocVersion).where(
        KnowledgeDocVersion.doc_id == doc_id,
        KnowledgeDocVersion.version == target_version))).scalar_one()
    doc.content = snapshot.content
    doc.version += 1 # 回退本身创建新版本 (类似 git revert)
    db.add(KnowledgeDocVersion(doc_id=doc_id, version=doc.version,
        content=snapshot.content,
        operated_by=f"rollback from v{target_version}"))
```

6. 跨端适配方案

平台	策略
Web (PC)	全功能桌面版 — 完整数据表格、虚拟滚动、地图聚合、实时 WebSocket
Web (Mobile)	TailwindCSS 响应式断点 (sm/md/lg) — 简化表单卡片、底部导航
PWA	Service Worker + Cache API — 离线缓存草稿、后台同步
企业微信/钉钉	独立适配 — 扫码录入商户、简化审批流程
Flutter (规划)	跨平台原生 — 共享 API 层、离线优先数据库

7. 后端：多维度表单数据处理

需求："多维度表单数据处理 | 查询过滤器 | 埋点日志 | 异常处理"

```
# 单路由多模型 - 四类表单共享查询逻辑
MODEL_MAP = {
    "merchant": MerchantForm, "listing": ListingForm,
    "product": ProductForm, "report": ReportForm,
}

async def query_forms(form_type, filters, db):
    model = MODEL_MAP[form_type]
    query = select(model)
    if filters.status:      query = query.where(model.status == ...)
    if filters.keyword:    query = query.where(model.name.ilike(...))
    if filters.geohash_prefix: query = query.where(model.geohash.startswith(...))
    # 统一分页 + 排序
    return query.order_by(...).offset(...).limit(...)

# 全局异常处理
@app.exception_handler(Exception)
async def global_exception_handler(request, exc):
    return JSONResponse(status_code=500, content={
        "error": "internal_error",
        "detail": str(exc) if settings.debug else "Internal error",
        "path": str(request.url.path),
    })
```

8. 后端：四类表单提交流程

需求："构建如下类型的表单提交流程：商户信息 | 商户房源 | 商户商品 | 商户报表"

```
# 四条独立端点 + 各自 Pydantic Schema (含示例值, Apifox 可导入)
POST /api/v1/forms/merchant/submit
  → MerchantSubmit { name, address, lat, lng, category }
  → 自动 geo (WKTElement) + geohash 编码 + 自动聚类

POST /api/v1/forms/listing/submit
  → ListingSubmit { merchant_id, title, area(>0), price(>0) }

POST /api/v1/forms/product/submit
  → ProductSubmit { merchant_id, name, sku, price(>0), stock(>=0) }

POST /api/v1/forms/report/submit
  → ReportSubmit { merchant_id, report_type(daily|weekly|monthly), period, data }

# 提交流程 (商户为例)
form = MerchantForm(
    name=body.name, address=body.address,
    geo=WKTElement(f"POINT({body.lng} {body.lat})", srid=4326),
    geohash=geohash_encode(lat, lng, 9),
    status=FormStatus.SUBMITTED)
db.add(form); await db.flush()
_log_submission(db, "merchant", str(form.id), "success", 0, None)
cluster_info = await _auto_cluster_new_merchant(db, form, lat, lng)
await db.commit()
return {"id": form.id, "status": "submitted", **cluster_info}
```

9. 后端：按类型分发的批处理流程

需求：“完成上述业务的批处理流程构建”

```
# 统一入口 → 按类型分发到不同业务处理函数
@router.post("/{form_type}/batch")
async def batch_process_forms(form_type, form_ids, db):
    rows = (await db.execute(select(model).where(model.id.in_(ids)))).all()
    if form_type == "merchant": results = _batch_merchant(...)
    if form_type == "listing":  results = _batch_listing(...)
    if form_type == "product":  results = _batch_product(...)
    if form_type == "report":   results = _batch_report(...)
    await notify_batch_progress(bid, total, total, "completed")
```

四种批处理流程对比

类型	步骤	校验规则	失败→
商户	①清洗 ②编码 ③批次 ④审批	名称≥2字符, geo有效	REJECTED
房源	①聚合 ②单价校验 ③审批	¥10~50,000/m²	REJECTED
商品	①SKU查重 ②库存 ③上架	SKU唯一, stock>0	REJECTED
报表	①校验 ②归档 ③汇总	data含"revenue"	REJECTED

WebSocket 每 10% 推送一次，状态文字随类型变化。

10. 测试体系：41 个自动化测试

层	框架	数量	覆盖
纯函数	pytest	17	Geohash(6) + Haversine(5) + 聚类(6)
API 集成	pytest+httpx	10	全字段、分页、过滤、聚合、WebSocket
前端	vitest	14	formatCell(5)+statusColors(3)+LRU(6)

```
# 纯函数验证物理准确性
def test_known_distance_reference():
    dist = haversine_m(32.041, 118.784, 32.060, 118.781) # 新街口→鼓楼
    assert 1900 < dist < 2300 # ~2.1km ±5%

def test_three_points_chain_clusters():
    pts = [(1,32.000,118.500),(2,32.003,118.500),(3,32.006,118.500)]
    assert len(cluster_by_distance(pts, 500)) == 1 # 链式连通

# API 集成 - WebSocket 握手验证
def test_websocket_endpoint_accepts_connection():
    s.sendall(b"GET /ws/progress HTTP/1.1...")
    assert "101" in s.recv(4096).decode()

# 运行
cd backend && source .venv/bin/activate && pytest tests/ -v # 27
cd frontend && npx vitest run # 14
```

附录：项目结构

```
B-demo/
├── docker-compose.yml          # PostgreSQL + Redis + Backend
├── apifox-openapi.json        # Apifox 导入用 OpenAPI 规范
├── README.md
├── backend/
│   └── app/
│       ├── main.py            # FastAPI 入口 + 全局异常处理
│       ├── models.py          # 7 个 ORM 模型
│       └── routes/
│           ├── forms.py        # 提交 + CRUD + 批处理 + 自动聚合
│           ├── aggregation.py  # Geohash + Haversine + 连通分量聚类
│           ├── analytics.py    # 成功率统计
│           ├── knowledge.py    # 文档 CRUD + 版本回退
│           └── ws.py           # WebSocket 广播
├── frontend/
│   └── src/
│       ├── store/index.ts     # Zustand MVVM store
│       └── app/
│           ├── forms/page.tsx # 虚拟滚动 + LRU 缓存
│           ├── aggregation/   # 地理聚合页面
│           ├── analytics/     # 成功率统计页面
│           └── knowledge/     # 文档 CRUD + 回退
└── backend/tests/
    ├── test_aggregation_pure.py # 17 纯函数测试
    └── test_api.py              # 10 API 集成测试
```